HCL-ASSIGNMENT - 2

Name: V. Jabivulla
USN :23BTRCL237
ID : HCLTFP2316135
Department :AIML

Question 1 : Database Design and Advanced Queries

a. Display employee names, department names, and manager names using joins.

```
57
58
59  SELECT
60      e.EmployeeName AS Employee,
61      d.DepartmentName AS Department,
62      m.EmployeeName AS Manager
63  FROM Employees e
64  JOIN Departments d
65      ON e.DepartmentID = d.DepartmentID
66  LEFT JOIN Employees m
67      ON e.ManagerID = m.EmployeeID;
68
69
70
```

b. Find employees earning more than their department's average salary.

```
68
69  SELECT
70      e.EmployeeName,
71      d.DepartmentName,
72      e.Salary
73  FROM Employees e
74  JOIN Departments d
75      ON e.DepartmentID = d.DepartmentID
76  WHERE e.Salary > (
77      SELECT AVG(e2.Salary)
78      FROM Employees e2
79      WHERE e2.DepartmentID = e.DepartmentID
80  );
81
82
83
84      CREATE TABLE Sales(
```

c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST.

```
81
82  SELECT MAX(Salary) AS SecondHighestSalary
83  FROM Employees
84  WHERE Salary < (
85      SELECT MAX(Salary)
86      FROM Employees
87  );
88
```

Output | Result #12

SecondHighestSalary
75000.00

d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff.

```
87  );
88
89  SELECT
90      e.EmployeeName,d.DepartmentName,e.Salary,
91      rnk
92  FROM (
93      SELECT
94          EmployeeID,
95          EmployeeName,
96          DepartmentID,
97          Salary,
98          RANK() OVER (
99              PARTITION BY DepartmentID
100             ORDER BY Salary DESC
101         ) AS rnk
102     FROM Employees
103 ) e
104 JOIN Departments d
105     ON e.DepartmentID = d.DepartmentID
106 WHERE rnk <= 3;
107
108
```

Question 2: Sales Analytics and Query Optimization

a. Calculate total revenue for each product.

```
139
140  SELECT ProductName, SUM(Quantity * UnitPrice) AS TotalRevenue
141  FROM Sales
142  GROUP BY ProductName;
143
144
```

Output | Result #18

ProductName	TotalRevenue
Laptop	420000.00
Mouse	12500.00
Keyboard	15000.00
Monitor	72000.00

b. Calculate monthly revenue and month-over-month revenue change using a window function.

```
143
144  SELECT
145      FORMAT(SaleDate, 'yyyy-MM') AS Month,
146      SUM(Quantity * UnitPrice) AS MonthlyRevenue,
147      SUM(Quantity * UnitPrice)
148          - LAG(SUM(Quantity * UnitPrice)) OVER (ORDER BY FORMAT(SaleDate, 'yyyy-MM')) AS MoMChange
149  FROM Sales
150  GROUP BY FORMAT(SaleDate, 'yyyy-MM')
151  ORDER BY Month;
152
153
154
```

c. Rewrite the following query to improve the potential for index usage and explain why your version may perform better.

```
151  ORDER BY Month;
152
153  SELECT *
154  FROM Employees
155  WHERE JoiningDate >= '2025-01-01'
156    AND JoiningDate < '2026-01-01';
157
158
159
160
161
```

Question 3: SQL Troubleshooting

a. Explain how an execution plan helps identify expensive operations in a query.

Ans).An execution plan shows how SQL Server processes a query step by step.

Operators used

(e.g., table scans, index seeks, joins, sorts).

It also reveals whether indexes are being used effectively or ignored.

b. Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning.

Ans).DepartmentID is a selective filter, so placing it first in the index improves lookups.

. Salary as the second column allows efficient range queries (e.g., employees with Salary > 50,000 in a department)

c. Write a query to identify duplicate customer email addresses and display only addresses occurring more than once.

```
157
158   SELECT Email, COUNT(*) AS Occurrences
159   FROM Customers
160   GROUP BY Email
161   HAVING COUNT(*) > 1;
162
163
164
165
```

SECTION B: LINUX

Question 4: Log Analysis

a. Find all lines containing ERROR.

```
EOF
[ec2-user@ip-172-31-44-169 ~]$ grep "ERROR" /var/log/application.log
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 09:30:15 ERROR payment-service Payment timeout
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:25:30 ERROR auth-service Login failed
2026-10-28 11:30:00 ERROR payment-service Database connection failed
[ec2-user@ip-172-31-44-169 ~]$
```

i-0fe83684a241a8d81 (awspractice)

b. Display the 20 most recent error entries in the file.

```
[ec2-user@ip-172-31-44-169 ~]$ grep "ERROR" /var/log/application.log | tail -n 20
2026-10-28 08:15:23 ERROR payment-service Transaction failed
2026-10-28 09:30:15 ERROR payment-service Payment timeout
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:25:30 ERROR auth-service Login failed
2026-10-28 11:30:00 ERROR payment-service Database connection failed
[ec2-user@ip-172-31-44-169 ~]$
```

c. Count the total number of error entries.

```
[ec2-user@ip-172-31-44-169 ~]$ grep -c "ERROR" /var/log/application.log
5
[ec2-user@ip-172-31-44-169 ~]$
```

d. Extract error entries for a specified date into a separate file.

```
[ec2-user@ip-172-31-44-169 ~]$ grep "^2026-10-28.*ERROR" /var/log/application.log > errors-2026-10-28.log
[ec2-user@ip-172-31-44-169 ~]$ grep "ERROR" /var/log/application.log | awk '{print substr($2,1,2)}' | sort | uniq -c
```

e. Explain how you would calculate the number of errors occurring in each hour

```
[ec2-user@ip-172-31-44-169 ~]$ grep "ERROR" /var/log/application.log | awk '{print substr($2,1,2)}' | sort | uniq -c
      1 08
      1 09
      2 10
      1 11
[ec2-user@ip-172-31-44-169 ~]$
```

Question 5: Permissions and System Troubleshooting

a. Create a group named finance and a report file, assuming you have the required privileges.

```
[ec2-user@ip-172-31-44-169 ~]$ getent group finance
finance:x:1001:
[ec2-user@ip-172-31-44-169 ~]$
```

b. Set permissions so the owner can read and write, the finance group can read, and other users have no access.

```
[ec2-user@ip-172-31-44-169 ~]$ sudo chown ec2-user:finance report.txt
chmod 640 report.txt
ls -l report.txt
-rw-r-----. 1 ec2-user finance 0 Oct 28 07:27 report.txt
[ec2-user@ip-172-31-44-169 ~]$
```

c. Identify the processes consuming the most CPU.

```
[ec2-user@ip-172-31-44-169 ~]$ ps aux --sort=-%cpu | head
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  1.9 173924 17872 ?        Ss   Oct25   0:16 /usr/lib/systemd/systemd --switched-root --system --deserialize=32
root           2  0.0  0.0      0     0 ?        S    Oct25   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    Oct25   0:00 [pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   Oct25   0:00 [kworker/R-rcu_gp]
root           5  0.0  0.0      0     0 ?        I<   Oct25   0:00 [kworker/R-sync_wq]
root           6  0.0  0.0      0     0 ?        I<   Oct25   0:00 [kworker/R-kvfree_rcu_reclaim]
root           7  0.0  0.0      0     0 ?        I<   Oct25   0:00 [kworker/R-slub_flushwq]
root           8  0.0  0.0      0     0 ?        I<   Oct25   0:00 [kworker/R-netns]
root          10  0.0  0.0      0     0 ?        I<   Oct25   0:00 [kworker/0:0H-events_highpri]
[ec2-user@ip-172-31-44-169 ~]$
```

d. Display memory usage and filesystem disk utilization.

```
[ec2-user@ip-172-31-44-169 ~]$ free -h
               total        used        free      shared  buff/cache   available
Mem:           913Mi       173Mi       456Mi       0.0Ki       283Mi       607Mi
Swap:             0B          0B          0B
[ec2-user@ip-172-31-44-169 ~]$
```

```
[ec2-user@ip-172-31-44-169 ~]$ df -h
Filesystem        Size  Used Avail Use% Mounted on
devtmpfs          4.0M     0  4.0M   0% /dev
tmpfs             457M     0  457M   0% /dev/shm
tmpfs             183M  444K  183M   1% /run
efivarfs          128K  2.7K  121K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p1    8.0G  1.6G  6.4G  21% /
tmpfs             457M     0  457M   0% /tmp
/dev/nvme0n1p128   10M  1.3M  8.7M  13% /boot/efi
tmpfs              92M     0   92M   0% /run/user/1000
[ec2-user@ip-172-31-44-169 ~]$
```

e. Find files larger than 100 MB under a specified application directory

```
[ec2-user@ip-172-31-44-169 ~]$ pwd
/home/ec2-user
[ec2-user@ip-172-31-44-169 ~]$ find /home/ec2-user/myapp -type f -size +100M
find: '/home/ec2-user/myapp': No such file or directory
```

SECTION C: PYSPARK DATA ENGINEERING

Question 7: PySpark DataFrame Operations

a. Display the schema and first five records.

[7]

```
df.printSchema()
df.show(5)
```

```
root
 |-- EmployeeID: integer (nullable = false)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: integer (nullable = true)
 |-- JoiningDate: date (nullable = true)

+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|Arjun Sharma|        IT| 75000| 2021-03-15|
|         2| Priya Patel|        HR| 58000| 2020-07-10|
|         3| Rohan Verma|   Finance| 90000| 2019-01-22|
|         4| Ananya Iyer|        IT| 62000| 2022-05-18|
|         5|  Amit Singh| Marketing| 45000| 2021-11-05|
+----------+------------+----------+------+-----------+
only showing top 5 rows
```

b. Filter employees whose monthly salary is greater than ₹60,000

```
[8]
0s
d = df.filter(col("Salary") > 60000)
d.show()
```

```
+----------+---------------+----------+------+-----------+
|EmployeeID|   EmployeeName|Department|Salary|JoiningDate|
+----------+---------------+----------+------+-----------+
|         1|   Arjun Sharma|        IT| 75000| 2021-03-15|
|         3|    Rohan Verma|   Finance| 90000| 2019-01-22|
|         4|    Ananya Iyer|        IT| 62000| 2022-05-18|
|         6|    Sneha Reddy|        HR| 65000| 2018-09-30|
|         7|Vikram Malhotra|   Finance|120000| 2015-04-12|
|         9|    Rahul Gupta| Marketing| 70000| 2020-02-20|
|        11|    Aditya Bose|   Finance| 85000| 2017-06-25|
|        12|     Pooja Nair|        IT| 95000| 2016-10-10|
|        14|    Meera Joshi|        HR| 72000| 2019-04-05|
+----------+---------------+----------+------+-----------+
```

c. Add an AnnualSalary column calculated from the monthly salary.

```
df_with_annual = df.withColumn("AnnualSalary", col("Salary") * 12)
df_with_annual.show(5)
```

```
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|         1|Arjun Sharma|        IT| 75000| 2021-03-15|      900000|
|         2| Priya Patel|        HR| 58000| 2020-07-10|      696000|
|         3| Rohan Verma|   Finance| 90000| 2019-01-22|     1080000|
|         4| Ananya Iyer|        IT| 62000| 2022-05-18|      744000|
|         5|  Amit Singh| Marketing| 45000| 2021-11-05|      540000|
+----------+------------+----------+------+-----------+------------+
only showing top 5 rows
```

d. Group employees by department and calculate the average salary and employee count.

```
dep= df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("EmployeeID").alias("EmployeeCount")
)
dep.show()
```

```
+----------+------------------+-------------+
|Department|     AverageSalary|EmployeeCount|
+----------+------------------+-------------+
|        HR|           61750.0|            4|
|   Finance|           88500.0|            4|
| Marketing|54333.333333333336|            3|
|        IT|           71750.0|            4|
+----------+------------------+-------------+
```

e. Display the five highest-paid employees.

```
top_paid = df.orderBy(col("Salary").desc()).limit(5)
top_paid.show()
```

```
+----------+---------------+----------+------+-----------+
|EmployeeID|   EmployeeName|Department|Salary|JoiningDate|
+----------+---------------+----------+------+-----------+
|         7|Vikram Malhotra|   Finance|120000| 2015-04-12|
|        12|     Pooja Nair|        IT| 95000| 2016-10-10|
|         3|    Rohan Verma|   Finance| 90000| 2019-01-22|
|        11|    Aditya Bose|   Finance| 85000| 2017-06-25|
|         1|   Arjun Sharma|        IT| 75000| 2021-03-15|
+----------+---------------+----------+------+-----------+
```

Question 8: Data Cleaning and Aggregation

a. Read the CSV file using an appropriate schema and header configuration.

```
from pyspark.sql import functions as F

schema = StructType([
    StructField("TransactionID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("TransactionDate", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True)
])

df = spark.read.csv("transactions.csv", header=True, schema=schema)
df.show()
```

```
+-------------+----------+---------------+-------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|
+-------------+----------+---------------+-------+--------+---------+
|            1|     101.0|     2024-01-10| Laptop|    NULL|  50000.0|
|            2|     102.0|     2024-01-12| Mobile|    NULL|  20000.0|
|            3|     103.0|     15/01/2024| Tablet|    NULL|  15000.0|
|            2|     102.0|     2024-01-12| Mobile|    NULL|  20000.0|
|            4|     104.0|        invalid| Laptop|    NULL|  50000.0|
|            5|      NULL|     2024-02-01| Tablet|    NULL|  15000.0|
|            6|     106.0|     2024-02-05|   NULL|    NULL|  10000.0|
|            7|     107.0|     2024-02-10| Mobile|    NULL|  20000.0|
+-------------+----------+---------------+-------+--------+---------+
```

b. Identify and remove duplicate transactions using a suitable business key

```
df_cleaned = df.dropDuplicates(["TransactionID"])
df_cleaned.show()
```

```
+-------------+----------+---------------+-------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|
+-------------+----------+---------------+-------+--------+---------+
|            1|     101.0|     2024-01-10| Laptop|    NULL|  50000.0|
|            2|     102.0|     2024-01-12| Mobile|    NULL|  20000.0|
|            3|     103.0|     15/01/2024| Tablet|    NULL|  15000.0|
|            4|     104.0|        invalid| Laptop|    NULL|  50000.0|
|            5|      NULL|     2024-02-01| Tablet|    NULL|  15000.0|
|            6|     106.0|     2024-02-05|   NULL|    NULL|  10000.0|
|            7|     107.0|     2024-02-10| Mobile|    NULL|  20000.0|
+-------------+----------+---------------+-------+--------+---------+
```

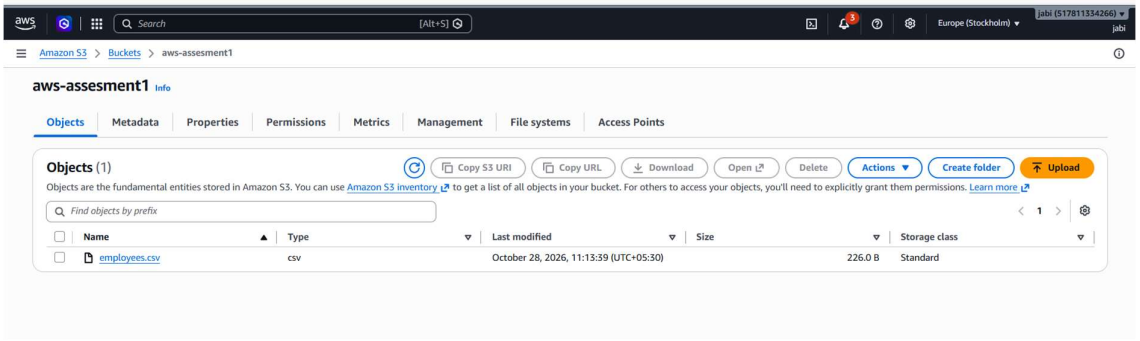
c. Handle missing values in required fields and explain your chosen approach.

```
df_cleaned = df_cleaned.dropna(subset=["TransactionID", "CustomerID", "Product"])
df_cleaned = df_cleaned.fillna({"Quantity": 1, "UnitPrice": 0.0})
df_cleaned.show()
```

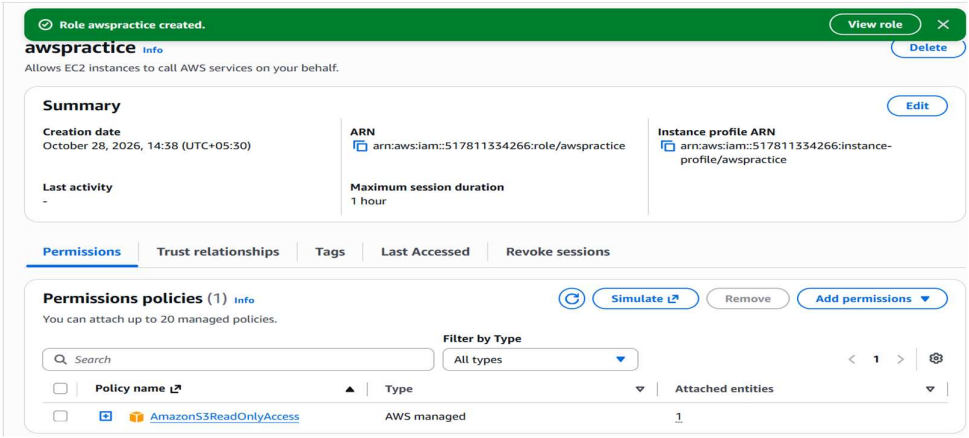
```
+-------------+----------+---------------+-------+--------+---------+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|
+-------------+----------+---------------+-------+--------+---------+
|            1|     101.0|     2024-01-10| Laptop|       1|  50000.0|
|            2|     102.0|     2024-01-12| Mobile|       1|  20000.0|
|            3|     103.0|     15/01/2024| Tablet|       1|  15000.0|
|            4|     104.0|        invalid| Laptop|       1|  50000.0|
|            7|     107.0|     2024-02-10| Mobile|       1|  20000.0|
+-------------+----------+---------------+-------+--------+---------+
```

Question 10: S3 and IAM Security

a. Create an S3 bucket with Block Public Access enabled and upload a sample file.



b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets.



c. Explain how you would verify that an unauthorized user cannot read the file.

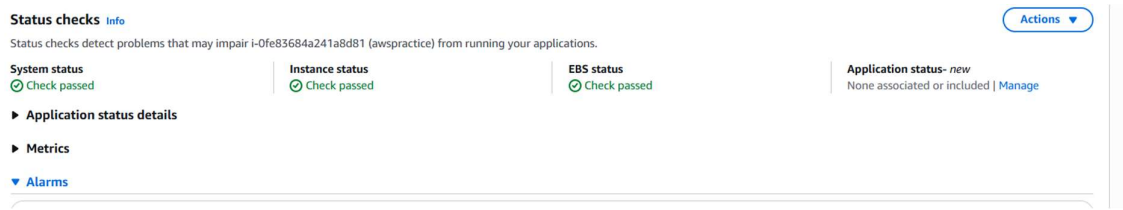
Ans).When user trys to open file, account that does not have required permissions.it shows access is denied and user is not able open the file.

d. Explain how encryption at rest protects the stored data.

Ans).it stores data in encrypted form on disk. If an attacker get access to storage, the data remains unreadable with out correct decription word.

Question 11: EC2 Troubleshooting and CloudWatch

a. Check the EC2 instance state and status checks.



b. Explain how security groups and route tables can affect application connectivity.

Ans).Security groups filter traffic at the EC2 instance level, while route tables define the network path. Together, they ensure that only authorized traffic reaches the right destination.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.

Ans).To configure a CloudWatch alarm for CPU utilization exceeding 85%, you need to define a metric, set a threshold, and specify actions.

Cloud watch -> create a alarm -> select cpu utilization ->define threshold and evaluation ->configure alarm actions -> Name and create alarm.

d. Identify one method for investigating the root cause of high CPU utilization.

Ans).One effective method for investigating the root cause of high CPU utilization is to analyze detailed performance metrics and logs from the instance.

Cloud watch -> detailed monitoring -> instance level investigation -> network and storage

Question 12: Event-Driven Processing

a. Identify a suitable service for executing the processing code.

Ans). Aws Lambda is the best choice. It allows you to run code without provisioning servers, scales automatically, and integrates tightly with S3 events.

b. Explain how an S3 upload can trigger processing.

Ans). You configure an S3 Event Notification on the bucket. When a new CSV file is uploaded (e.g., PUT event), S3 sends an event to Lambda, which automatically invokes the function to validate and process the file.

c. Explain the IAM permissions needed to read the input file and write the results.

Ans). s3:GetObject permission to read the uploaded CSV file from the input bucket.

s3:PutObject permission to write the processed results to the output bucket or location. Optionally, s3:ListBucket if listing objects is required.

d. Explain how errors and processing logs can be captured.

Ans). Amazon CloudWatch Logs automatically captures Lambda execution logs, including errors and custom logging statements.

You can also configure CloudWatch Alarms or DLQs (Dead Letter Queues) with Amazon SQS/SNS to handle failed processing events.

e. Explain how to prevent duplicate processing or recursive triggering.

Ans). Use separate input and output buckets (or distinct prefixes) so that writing results does not trigger the same Lambda again.

Add object metadata or DynamoDB tracking to ensure each file is processed only once.

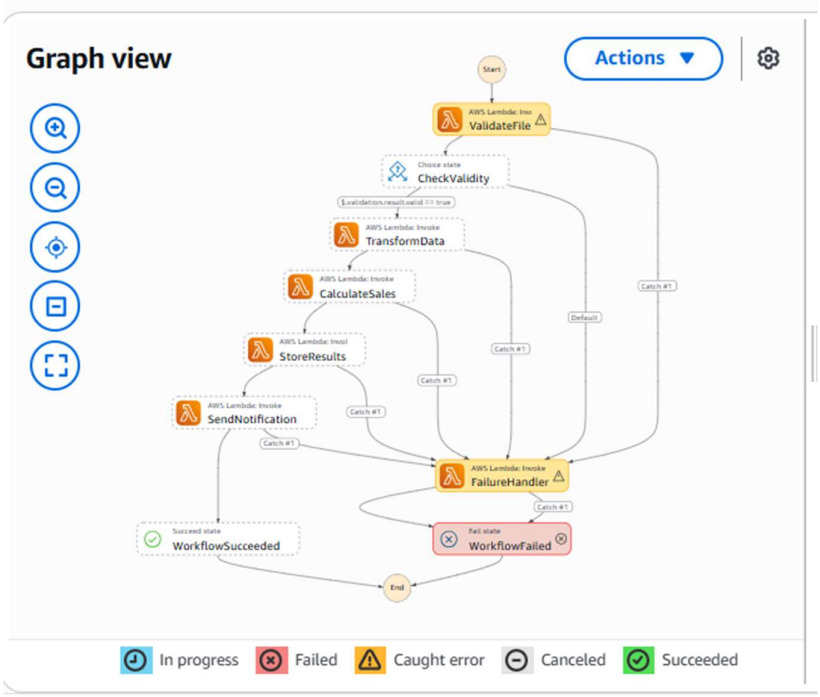
Configure event filters in S3 to trigger only on specific prefixes or suffixes (e.g., .csv).

SECTION E: AWS STEP FUNCTIONS

Question 13: State Machine Design

a. Identify suitable states for validation, processing, storage, and notification

Ans). Suitable States: Use Task states to run AWS Lambda functions or AWS SDK integrations for validation, data transformation/calculation, S3 upload, and SNS notification; optionally add a Choice state after validation for branching and a Fail or Succeed state at termination points.



b. Explain how a Choice state can route invalid files to a failure path and valid files to processing.

Ans). Choice State Routing: A Choice state inspects a boolean or status field in the validation output (e.g., $.isValid == true) to route valid files to the processing Task state and invalid files via Default or matching condition directly to a Fail state.

c. Explain how Retry and Catch can handle transient failures and permanent errors.

Ans). Retry vs. Catch: Retry automatically re-executes a failed state with configurable intervals and backoffs to overcome temporary glitches like network timeouts, while Catch

intercepts unrecoverable or persistent errors to cleanly divert execution to a fallback or error-handling state.

d. Explain how the state machine can invoke Lambda functions or other supported AWS services.

Ans). Invoking Lambda & AWS Services: The state machine invokes services using a Task state by specifying the target service's ARN in the "Resource" field (such as a Lambda ARN or an optimized service integration like arn:aws:states:::sns:publish)

e. Explain how execution history helps troubleshoot a failed workflow.

Ans). Execution History Troubleshooting: The execution history logs a chronological step-by-step audit of state entries, exits, timestamps, exact input/output payloads, and detailed error tracebacks to quickly pinpoint the exact failure point.

Question 14: Step Functions Implementation

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary.

Ans). Sequential execution: AWS Step Functions executes ValidateFile, TransformData, and LoadResults in sequence using Task states.

b. Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts.

Ans). Validation: A Choice state checks validation results. If the file is invalid, the workflow stops using a Fail state.

c. Configure a Catch transition for the loading task that routes to an error-handling state.

Ans). Retry: The TransformData task uses Retry with IntervalSeconds: 2, MaxAttempts: 3, and BackoffRate: 2 to handle temporary failures.

d. Explain how data from one task is passed to the next task.

Ans). Catch: The LoadResults task uses Catch to route permanent errors to RecordFailure for logging.

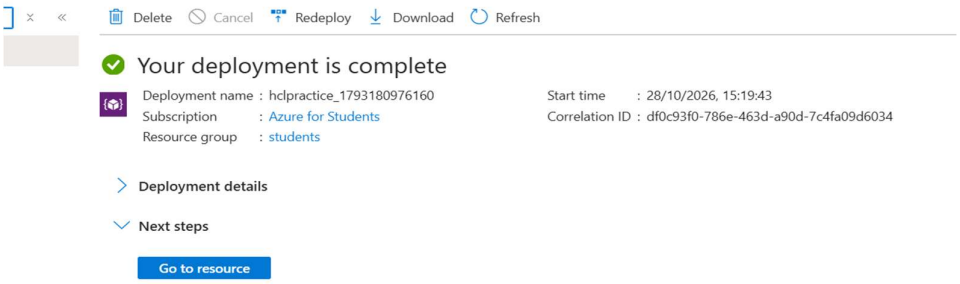
e. Explain how you would test successful and failed executions.

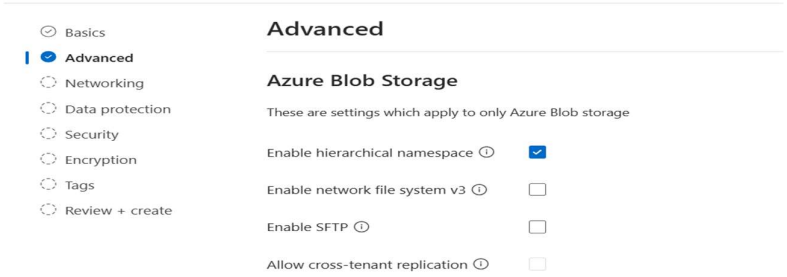
Ans). Data passing and testing: Payload.$ passes input between tasks, while ResultPath stores task outputs. Execution history helps debug errors and verify successful or failed runs.

SECTION F: MICROSOFT AZURE

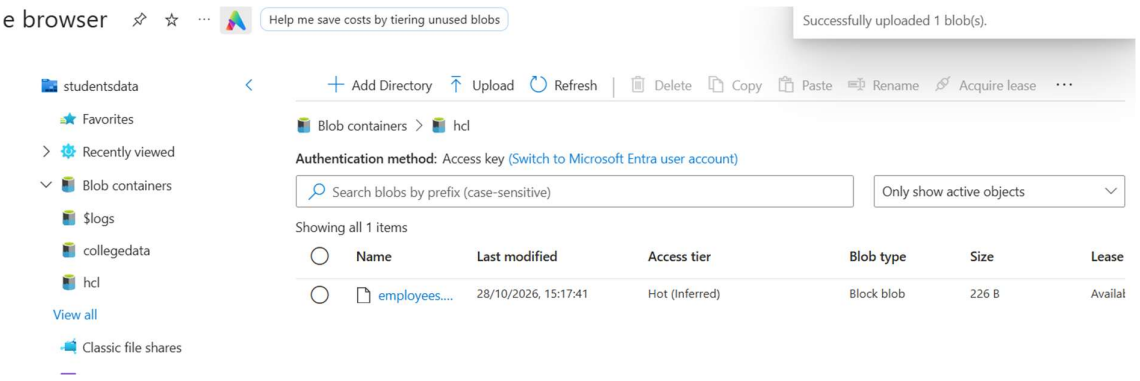
Question 15: Azure Data Lake Storage and Security

a. Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2.





b. Create a filesystem or container and upload a sample dataset.



c. Explain how Microsoft Entra ID and Azure role-based access control can manage access to the data.

Ans). **Microsoft Entra ID (formerly Azure AD)** provides centralized identity management. Every user or application must authenticate through Entra ID before accessing the data.

Role-Based Access Control (RBAC) assigns fine-grained permissions. Instead of granting blanket access, you assign roles such as *Storage Blob Data Reader* (read-only), *Storage Blob Data Contributor* (read/write), or *Storage Blob Data Owner* (full control).

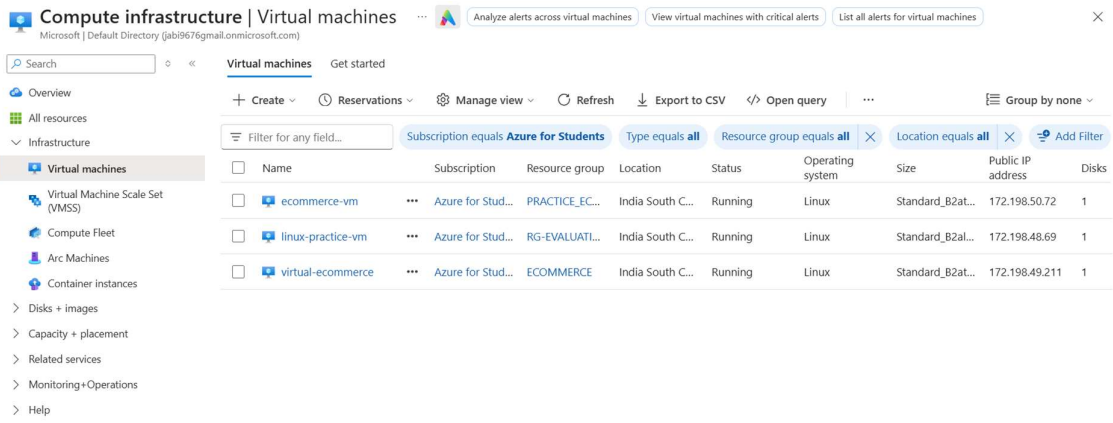
d. Explain how you would verify that unauthorized users cannot access the dataset.

Ans).Attempt to access the dataset using an account without RBAC permissions — access should be denied with an authorization error.

Review Azure Activity Logs and Access Diagnostics to confirm that unauthorized requests are blocked.

Question 16: Azure VM Troubleshooting

a. Verify the VM's provisioning and power states.



b. Explain how Network Security Group rules can prevent connectivity.

Ans).NSG rules may block connectivity if inbound traf ic is denied or the required port is not allowed.

c. Explain how subnet and routing con iguration affect connectivity

Ans). Incorrect subnet settings, route tables, or User De ined Routes (UDRs) can prevent traf ic from reaching the VM. Check that the subnet is correctly con igured and that routes allow traf ic to reach the destination.

d. Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet.

Ans). Use Azure Bastion to connect securely to the VM through the Azure Portal without exposing the VM's public IP address. And Use Network Watcher tools, such as IP low verify and connection troubleshoot, to identify blocked ports and connectivity issues.

Question 17: Azure Data Integration

a. Select an appropriate service to orchestrate data movement.

Ans). Use Azure Data Factory to orchestrate and automate data movement and processing.

b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2.

Ans).Create a Copy activity in Azure Data Factory to read the CSV ile from the source and copy it into Azure Data Lake Storage Gen2 (ADLS Gen2)

c. Describe how to organize files by ingestion date.

Ans).Store iles in date-based folders,

for example:/transactions/year=2026/month=10/day=28/

d. Explain how to handle malformed records and missing values.

Ans). Use Azure Data Factory mapping data lows or data validation steps to identify malformed records and handle missing values. Store invalid records in a separate error folder for further review.

e. Explain how to monitor pipeline execution and validate data completeness.

Ans). Use Azure Data Factory Monitor to track pipeline execution and errors. Validate data completeness by comparing source and destination record counts and checking for missing or duplicate records.